

Forward Private Searchable Symmetric Encryption with Optimized I/O Efficiency

- **背景与动机**

- 近年来，针对可搜索加密的实际攻击揭露了安全性隐患，其中前向隐私成为防范适应性泄漏攻击的关键属性。
- 传统上，实现前向隐私常常伴随效率低下的问题，因此大多数现有方案并不支持这一特性。

- **现有方法的不足**

- Bost 在 2016 年提出了一种前向隐私方案，该方案在通信开销上与攻击防护均表现优秀，但使用了较为低效的公钥加密技术，并且 I/O 性能较差，限制了其实用性和在大规模数据环境下的扩展性。

- **提出的新方案**

- 作者提出了两个新方案：**FAST** 与 **FASTIO**。
- **FAST**：借助对称加密原语实现了前向隐私，同时保持了与 Bost 方案相同的通信效率，从而大幅提高计算效率。
- **FASTIO**：在 FAST 基础上进一步改进了 I/O 性能，优化了整体运行效率。

- **实验结果**

- 通过实验对比，新方案（FAST 及 FASTIO）均显示出比 Bost 方案更高的效率，证明了在实际应用中具备较好的竞争力。

对没实现前向安全方案的攻击：

- **攻击流程概述**：服务器首先构造一组包含特定关键词的文件，并将这些文件发送给客户端，从而诱使客户端对这些文件进行加密。
- **攻击利用**：加密后的文件上传后，服务器利用之前客户端提交的检索令牌，在这些注入的文件中进行搜索。
- **攻击结果**：借助已知注入文件中包含哪些关键词及匹配令牌的情况，服务器能够轻易推断出令牌所加密的关键词。

前向安全：新插入的文件不能与之前的查询关联

实现前向安全的简单方法：当需要添加新文件时，客户端从服务器下载所有加密文件，用新文件重新索引它们，并用新密钥重新加密所有内容。

Bost 的方案 Sophos

- **关键词状态维护与更新：**

客户端为每个关键词维护一个状态，每当包含该关键词的文件更新时，会对状态进行更新。更新后的状态用来生成新的加密索引，意味着之前的令牌将失效。

- **确保前向隐私：**

由于每次文件更新都会改变关键词状态，旧的令牌无法匹配到新生成的索引，从而保护了前向隐私。

- **状态令牌生成与服务器交互：**

状态在客户端保密，直到用户进行搜索时，客户端才生成一个包含当前状态的恒定大小的令牌。

这个令牌不仅代表了当前的状态，还使服务器能够推导出所有历史状态，从而能够对所有曾经更新的文件进行匹配搜索。

缺陷：

1. 该方案基于公钥密码学，如果数据频繁更新，公钥操作将成为主要的性能瓶颈。如果只使用对称加密原语，那将是理想的。
2. 搜索操作的I/O效率不高。I/O已成为阻止可搜索加密**扩展到大数据**的主要瓶颈。在Sophos中，搜索操作的局部性（衡量I/O效率的指标）随更新次数线性增加。

本文的两个方案

1. **FAST**：其想法来源于单链表，将**更新**看作列表中的节点，其包含一个临时密钥，**密钥相当于指针**，指向前一个节点。通过对前一个状态用临时密钥加密来产生新的状态。服务器可以利用当前临时密钥对当前状态进行解密，从而获得之前的状态。然后，借助新的临时密钥可以依次向历史方向追溯所有更新。通过这种方式，服务器能够遍历整个状态链，从而检索到所有历史更新的数据。

- **前向隐私保证：**

关键在于每次产生的新临时密钥均是由客户端随机生成且与此前密钥无关。这样，即使服务器通过回溯已知状态和密钥，也无法预测客户端下一次更新时使用的密钥或状态，从而有效实现前向隐私。

2. **FASTIO (I/O Optimized)**

虽然 FAST 使用纯对称密钥加密方法提高了计算效率，但其设计带来了一些 I/O（输入/输出）效率的问题，主要体现在：

- **临时密钥存储开销：** 为了保证状态链的完整性，服务器必须存储所有更新中产生的临时密钥。
- **非局部数据访问：** 为了跟踪整个状态链，服务器需要进行多次非连续的存储读取操作，这在大量数据更新下显著增加了 I/O 开销。

FASTIO 的优化思路： 每次搜索查询后，服务器已经得知了搜索结果，于是允许服务器将该搜索结果存储起来。有了搜索结果作为缓存后，服务器只需要维护最新的状态，无需再存储所有的临时密钥。这样可以大幅减少服务器端的存储需求。

- **减少非局部读取：**

由于之前的搜索结果已经连贯存储，再次访问时可以连续读取，避免了频繁的随机访问，从而提升了 I/O 效率。

FAST方案

大致想法

每次状态更新时，都是由客户端随机选择一个临时密钥，用此临时密钥加密当前状态得到一个新的状态。新的状态不暴露给服务器，而临时密钥**不直接**暴露给服务器。

在进行搜索时，客户端只需将当前状态发送给服务器，服务器利用该状态先解密出对应的临时密钥，再用这个密钥对当前状态进行解密，从而恢复出前一个状态。

这个过程可以迭代进行，直到恢复出所有之前的状态，这样服务器就可以构建出完整的状态链来完成搜索操作。

Setup ($\lambda, \perp; \perp$) <i>Client:</i> 1: $k_s \xleftarrow{\$} \{0, 1\}^\lambda$ 2: $\Sigma \leftarrow$ empty map <i>Server:</i> 3: $T \leftarrow$ empty map Update ($k_s, \Sigma, ind, w, op; T$) <i>Client:</i> 4: $t_w \leftarrow F(k_s, h(w))$ 5: $(st_c, c) \leftarrow \Sigma[w]$ 6: if $(st_c, c) = \perp$ then 7: $st_0 \xleftarrow{\$} \{0, 1\}^\lambda, c \leftarrow 0$ 8: end if 9: $k_{c+1} \xleftarrow{\$} \{0, 1\}^\lambda$ 10: $st_{c+1} \leftarrow P(k_{c+1}, st_c)$	11: $\Sigma[w] \leftarrow (st_{c+1}, c + 1)$ 12: $e \leftarrow (ind op k_{c+1}) \oplus H_2(t_w st_{c+1})$ 13: $u \leftarrow H_1(t_w st_{c+1})$ 14: send (u, e) to server <i>Server:</i> 15: $T[u] = e$ Search ($k_s, \Sigma, w; T$) <i>Client:</i> 16: $t_w \leftarrow F(k_s, h(w))$ 17: $(st_c, c) \leftarrow \Sigma[w]$ 18: if $(st_c, c) = \perp$ then 19: return $\emptyset$ 20: end if 21: send (t_w, st_c, c) to Server <i>Server:</i> 22: $ID, \Delta \leftarrow \emptyset$	23: for $i = c$ to 1 do 24: $u \leftarrow H_1(t_w st_i)$ 25: $e \leftarrow T[u]$ 26: $(ind, op, k_i) \leftarrow e \oplus H_2(t_w st_i)$ 27: if $op = \text{"del"}$ then 28: $\Delta \leftarrow \Delta \cup \{ind\}$ 29: else if $op = \text{"add"}$ then 30: if $ind \in \Delta$ then 31: $\Delta \leftarrow \Delta - ind$ 32: else 33: $ID \leftarrow ID \cup \{ind\}$ 34: end if 35: end if 36: $st_{i-1} \leftarrow P^{-1}(k_i, st_i)$ 37: end for 38: send ID to client
--	--	--

Fig. 1: Pseudocode of Protocols in FAST

Setup

- **客户端:**
 - 生成一个 λ 位长的长期密钥 k_s ，该密钥用于对关键词进行加密处理，防止服务器自行生成合法的令牌。
 - 初始化空映射 Σ 用来存储每个关键词的最新状态。
- **服务器:**
 - 初始化一个空的映射 T ，用于存储更新时生成的加密索引条目。

Update

当客户端需要对包含关键词 w 的文件进行更新（可能是添加或删除操作）时，执行以下步骤：

(a) 状态的获取与初始化

- **关键词加密和映射索引:**
客户端通过伪随机函数 F 和哈希函数 h 将关键词 w 转换成索引值 t_w 。
- **状态与计数器:**
从本地状态存储 Σ 中查找关键词 w 的当前状态 st_c 和计数器 c （记录状态的版本）。
 - 若该关键词首次更新，则生成初始状态 st_0 。

(b) 状态演化与临时密钥生成

- **生成临时密钥:**
客户端生成一个随机的临时密钥 k_{c+1} 。
- **状态更新公式:**
使用伪随机排列 P 对前一个状态进行加密，得到新的状态：

$$st_{c+1} = P(k_{c+1}, st_c)$$

（这里可以理解为“用 k_{c+1} 对 st_c 进行加密”）

- **嵌入临时密钥:**
临时密钥 k_{c+1} 并不存储在客户端，而是嵌入到一个加密索引条目中。具体来说，将 k_{c+1} 以加密形式存储：

$$e \leftarrow (ind || op || k_{c+1}) \oplus H_2(t_w || st_{c+1})$$

这里的加密方式依赖于哈希函数 H_2 和异或操作，确保只有知道当前状态 st_{c+1} 的服务器才能解密出 k_{c+1} 。

- **生成引用值：**
客户端利用哈希函数 H_1 结合当前状态和关键词生成一个引用 u （用于在服务器端定位索引条目）。
- **更新服务器索引：**
客户端发送更新包 (e, u) 给服务器，服务器将该对映射存入 T 。
- **状态更新保存：**
客户端在本地状态映射 Σ 中更新关键词 w 的状态为 st_{c+1} 和计数器 $c + 1$ 。

Search

当客户端需要搜索关键词 w 时，流程如下：

(a) 客户端生成搜索令牌

- 客户端从 Σ 中获取关键词 w 的最新状态 st_c 和计数器 c 。
- 客户端使用长期密钥 k_s 加密关键词 w 生成搜索令牌 t_w （例如，令牌中可能包含加密后的 w 和其它必要的标识信息）。
- 将搜索令牌 (t_w, st_c, c) 发送给服务器。

(b) 服务器的状态回溯

服务器根据收到的最新状态 st_c 开始**向后回溯**所有历史状态，以便定位所有相关的更新记录：

1. 循环回溯过程：

对于当前状态 st_i （初始为 st_c ），服务器：

- 利用 st_i 在映射 T 中查找对应的加密条目 e 。
- 通过解密 e （依赖于当前状态 st_i ）恢复临时密钥 k_i ：

$$k_i = e \oplus H_2(t_w || st_i)$$

- 用恢复的 k_i 和逆伪随机排列 P^{-1} 还原前一个状态：

$$st_{i-1} = P^{-1}(st_i, k_i)$$

- 服务器重复上述步骤，直至回溯到初始状态 st_0 为止。

2. 处理更新操作：

- **添加操作 (Add)：**
如果某次更新为添加文件，服务器记录文件标识符 ind ；
- **删除操作 (Del)：**
如果更新为删除文件，服务器将对应的 ind 加入集合 Δ 。

在回溯过程中，服务器维护集合 Δ 来平衡“添加”和“删除”的更新。

- 如果在回溯时发现文件 ind 存在于 Δ 中，则说明该文件在之后的更新中被删除，最终不会出现在搜索结果中。

3. 生成搜索结果：

- 服务器根据所有回溯得到的更新记录，筛选出未被删除的文件标识符，形成最终的搜索结果。

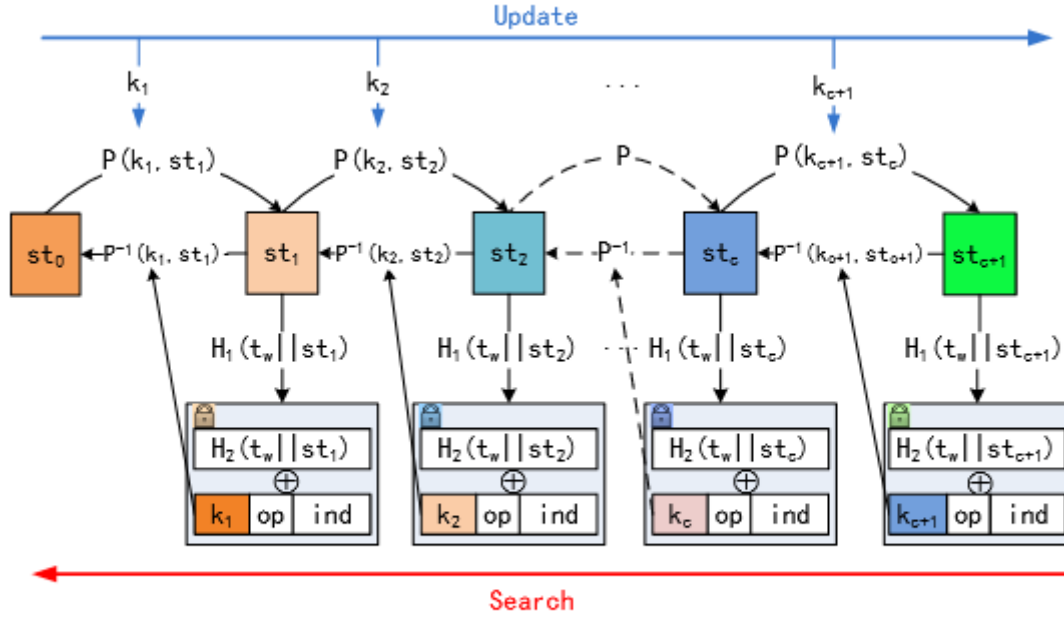


Fig. 2: Update and Search in FAST

复杂性分析

客户端

计算复杂度：更新与搜索都是 $O(1)$

存储复杂度：客户端只需要存储一个 λ 位的长期密钥 k_s 和一个状态映射 Σ ，其大小为 $O(|W|)$

服务器

计算复杂度：

- 更新： $O(1)$
- 搜索： $O(c_w)$ ， c_w 表示从系统初始化以来，包含关键词 w 的所有更新操作（包括添加和删除）的总数。

存储复杂度：服务器存储一个映射 T ，其大小为 $O(N_u)$ 。其中 $N_u = \sum_{w \in \mathcal{W}} c_w$ 表示自系统初始化以来，所有文档和关键词对更新的总数。因此，服务器的存储需求取决于系统中更新操作的累计数量。

通信复杂度：

- 更新操作的通信复杂度为 $O(1)$ ，即每次更新传输的数据量是固定的，不会随着系统规模扩大而增加。
- 搜索查询的通信复杂度为 $O(1)$ ，客户端只发送固定长度的搜索令牌（最新状态）。

返回的搜索结果集的通信复杂度为 $O(|DB(w)|)$ ，其中 $|DB(w)|$ 是匹配关键词 w 的文件数量。这表示结果数据的传输量与匹配的文件数成正比。

安全性分析

定理 1 Let F be a pseudorandom function, P be a pseudorandom permutation. Let H_1 and H_2 be two hash functions modeled as random oracles. Define leakage

$\mathcal{L} = (\mathcal{L}_{Setup}, \mathcal{L}_{Search}, \mathcal{L}_{Update})$ as

$$\begin{aligned}\mathcal{L}_{Setup} &= \perp \\ \mathcal{L}_{Search}(w) &= (\text{ap}(w), \text{qp}(w)) \\ \mathcal{L}_{Update}(i, \text{op}_i, w, \text{ind}_i) &= (i, \text{op}_i, \text{ind}_i)\end{aligned}$$

Then FAST is a $\mathcal{L}$ -adaptively-secure dynamic SSE with forward privacy.

FASTIO方案

Setup ($\lambda, \perp; \perp$) <i>Client</i> : 1: $k_s \xleftarrow{\$} \{0, 1\}^\lambda$ 2: $\Sigma \leftarrow \text{empty map}$ <i>Server</i> : 3: $T_e, T_c \leftarrow \text{empty map}$ Update ($k_s, \Sigma, \text{ind}, w, \text{op}; T_e$) <i>Client</i> : 4: $(st, c) \leftarrow \Sigma[w]$ 5: if $(st, c) = \perp$ then 6: $st \xleftarrow{\$} \{0, 1\}^\lambda$ 7: $c \leftarrow 0$ 8: end if 9: $u \leftarrow H_1(st (c + 1))$ 10: $e \leftarrow (\text{ind} \text{op}) \oplus H_2(st (c + 1))$ 11: $\Sigma[w] \leftarrow (st, c + 1)$	12: send (u, e) to server <i>Server</i> : 13: $T_e[u] = e$ Search ($k_s, \Sigma, w; T_e, T_c$) <i>Client</i> : 14: $(st, c) \leftarrow \Sigma[w]$ 15: if $(st, c) = \perp$ then 16: return $\emptyset$ 17: end if 18: $t_w \leftarrow F(k_s, h(w))$ 19: if $c \neq 0$ then 20: $k_w \leftarrow st, st \xleftarrow{\$} \{0, 1\}^\lambda$ 21: $\Sigma[w] \leftarrow (st, 0)$ 22: else 23: $k_w \leftarrow \perp$ 24: end if 25: send (t_w, k_w, c) to Server	<i>Server</i> : 26: $\text{ID} \leftarrow \emptyset$ 27: $\text{ID.add}(T_c[t_w])$ 28: if $k_w = \perp$ then 29: return ID 30: end if 31: for $i = 1$ to c do 32: $u_i \leftarrow H_1(k_w i)$ 33: $(\text{ind}, \text{op}) \leftarrow T_e[u_i] \oplus H_2(k_w i)$ 34: if $\text{op} = \text{"del"}$ then 35: $\text{ID} \leftarrow \text{ID} - \{\text{ind}\}$ 36: else if $\text{op} = \text{"add"}$ then 37: $\text{ID} \leftarrow \text{ID} \cup \{\text{ind}\}$ 38: end if 39: delete $T_e[u_i]$ 40: end for 41: $T_c[t_w] \leftarrow \text{ID}$ 42: send ID to client
--	--	--

Fig. 3: Pseudocode of Protocols in FASTIO

Setup 协议（初始化）

客户端（行 1-2）

1. 随机生成一个长期密钥 k_s （用于所有关键词的伪随机函数）。
2. 初始化空映射 Σ 。对每个关键词 w ，后续会在 $\Sigma[w]$ 中维护一对 (st, c) ：
 - st ：当前的“主状态”
 - c ：自上次搜索以来的更新计数器

服务器（行 3）

1. 初始化两个空映射：
 - T_e ：存储所有待处理的「更新令牌」 (u, e)
 - T_c ：缓存每个关键词上次搜索的完整结果集

与 FAST 的区别：FAST 只用一个映射存储所有更新，且每次更新都要重新演进状态；FASTIO 则用两张表分工，并且“主状态”只在搜索后更新一次，中间的多次更新由计数器管理。

Update 协议（增删文档）

当客户端要对关键词 w 做一次“插入”或“删除”操作 (ind, w, op) 时，按行 4-12 执行：

1. 取状态 $(st, c) \leftarrow \Sigma[w]$ 。
2. 若这是 w 的**首次更新或刚搜过一次**，则：
 - 随机重采样新主状态 $st \leftarrow \{0, 1\}^\lambda$ (行 6)
 - 将计数器 $c \leftarrow 0$ (行 7)
3. 用哈希生成本次更新的子状态

$$u = H_1(st \parallel (c + 1))$$

4. 用另一个哈希生成掩码，与操作及文档 ID 异或，得到

$$e = (ind \parallel op) \oplus H_2(st \parallel (c + 1))$$

5. 将更新令牌 (u, e) 发给服务器，服务器把它存入 $T_e[u]$ (行 12)。
6. 本地把计数器 $c \leftarrow c + 1$ ，写回 $\Sigma[w]$ (行 11)。

要点：两次搜索之间，主状态 st 保持不变，所有更新仅靠计数器 c 和子状态 u 区分。这样既保证了 unlinkability，又减少了每次都要重新采样主状态的开销。

Search 协议（关键词检索）

客户端发起关键词 w 的搜索，分为两部分：Token 构造（行 14-25）和服务器处理（行 26-42）。

3.1 客户端构造搜索令牌

1. **查表取值：** $(st, c) \leftarrow \Sigma[w]$ (行 14)。
2. 计算 $t_w = F(k_s, h(w))$ ，用来在服务器缓存 T_c 中定位上次结果 (行 18)。
3. 若 $c=0$ (自上次搜索后无更新)：
 - 只发送 t_w 即可 (行 25)，保持 st 不变。
4. 若 $c>0$ (有 c 条未处理更新)：
 - 发送 $(t_w, k_w = st, c)$ (行 25)
 - **立即本地重采样新主状态** $st \leftarrow \{0, 1\}^\lambda$ ，并令 $c \leftarrow 0$ (行 19-21)，以便下次更新继续 unlinkable。

3.2 服务器端增量更新

服务器收到令牌后（行 26-27）：

1. **取缓存结果：** 用 t_w 在 T_c 中查上次结果集 ID (行 27)。
2. 若客户端只发了 t_w (说明 $c=0$)，直接返回缓存的 ID (行 28-30)。
3. 否则 (有新更新)，依次处理 $i = 1 \dots c$ (行 31-40)：
 - 计算子状态 $u_i = H_1(k_w \parallel i)$ (行 32)。
 - 从 $T_e[u_i]$ 取出密文 e_i ，还原 $(ind, op) = e_i \oplus H_2(k_w \parallel i)$ (行 33)。
 - 按“add”/“del”将 ind 加入或移出结果集 ID (行 34-37)。
 - 删除该条 $T_e[u_i]$ (行 39)。

4. 将更新后的 ID 缓存回 $T_c[t_w]$ (行 41), 再返回给客户端 (行 42)。

增量效率: 服务器仅需处理自上次搜索以来的 c 条更新, 而不必每次都遍历整个索引。

4. 与 FAST 的主要对比

特性	FAST	FASTIO
状态更新	每次“增删”都重采样新主状态	只在“搜索”后重采样, 更新用计数器区分
服务器存储	一张表存所有更新令牌	两张表: T_e 存更新, T_c 缓存结果
搜索开销	每次搜索要扫描所有历史更新	只增量扫描自上次以来的更新 (或直接返回缓存)

5. 前向隐私与效率

- 前向隐私:** 一旦某条更新在搜索时被服务器消费 (T_e 中删除), 其内容以及对应的子状态就永不泄露给服务器。
- 高效搜索:**
 - 若无新更新, 搜索变成一次简单的缓存读取, 近乎零开销。
 - 若有更新, 仅按计数器 c 次迭代应用增量补丁, 大幅减少扫描量。

安全性分析

定理 2 Let F be a pseudorandom function, P be a pseudorandom permutation. Let H_1 and H_2 be two hash functions modeled as random oracles. Define leakage

$\mathcal{L} = (\mathcal{L}_{Setup}, \mathcal{L}_{Search}, \mathcal{L}_{Update})$ as

$$\begin{aligned}\mathcal{L}_{Setup} &= \perp \\ \mathcal{L}_{Search}(w) &= (\text{ap}(w), \text{qp}(w)) \\ \mathcal{L}_{Update}(i, \text{op}_i, w, \text{ind}_i) &= (i, \text{op}_i, \text{ind}_i)\end{aligned}$$

Then FASTIO is a $\mathcal{L}$ -adaptively-secure dynamic SSE with forward privacy.

IO效率分析

1. 服务器端索引大小与读效率接近最优

- FASTIO 在服务器上用两张表 T_e (存每条更新, 开销 $l + 1$ 比特) 和 T_c (存上次搜索结果, 开销为若干 l 比特的集合) 来组织索引。
- 若共有 N_u 个文档 / 关键词对, 整体索引大小介于 $N_u \cdot l$ 与 $N_u \cdot (l + 1)$ 比特之间。
- 每次搜索要读出 $|\text{DB}(w)|$ 个文档 ID——其中从 T_e 读取时每个 ID 多读 1 比特 (表示增删操作), 从 T_c 读取则无额外开销。
- 因此, 读入比特数在 $|\text{DB}(w)| \cdot l$ 到 $|\text{DB}(w)| \cdot (l + 1)$ 之间, 对应的读效率为 1 到 $1 + \frac{1}{l+1}$; 当 l 很大 (如 128) 时, 额外开销不足 1%。

2. 访问局部性 (locality) 与前向隐私的权衡

- 读 $\mathbf{T}_c$ 的上次搜索结果只需一次连续读；但对自上次搜索以来的 $\bar{c}_w$ 条新更新，仍需 $\bar{c}_w$ 次独立访问 $\mathbf{T}_e$ 。
- 整体的读取局部性是 $\bar{c}_w + 1$ （对比若不缓存则是 c_w ）。
- 如同已有研究所示，在保证前向隐私的前提下，最坏情况下的访问局部性无法进一步改善；如果某关键词很少被搜，缓存带来的优势也很有限。